

Полета и свойства на класове

Казахме, че Обектно-ориентираното програмиране ООП се опитва да представи обектите от реалния заобикалящ свят като класове. Класовете имат:

- 1) свойства (характеристики, атрибути, полета)
- 2) действия (методи, които изчисляват нещо и връщат резултат)

Нека създадем нов клас Person и се опитваме да го пресъздадем възможно най-реално:

```
0 references
internal class Person
{
    public int Id { get; set; }
}
```

Забелязваме, че по подразбиране, класът е internal (достъпен само в рамките на проекта). Класовете не могат да бъдат private или protected, защото техният смисъл е да бъдат достъпни в Main(). Също, имената на класовете са в единствено число – ние създаваме обекти по техния шаблон. Всеки един клас може да бъде използван като модел за таблица на База от данни БД, където имаме много записи на много обекти, затова е желателно винаги да започваме с ID, което е уникално за всеки нов обект. Това, обаче, ще го оставим за XII клас, когато изучаваме по-подробно БД и уеб-приложения. Другото, което забелязваме, е че по подразбиране, свойствата са public, с главна буква. Intellisense ни дописва автоматично get и set – с новия синтаксис, който включва скрита защита в себе си. Нека видим.

Автоматични свойства – с клавишна комбинация Prop + Tab + Tab извикваме snippet ->

```
0 references
internal class Person
{
    pro
    prop
    propa
    propda
    property
    Code snippet for 'property'
```

И дописва първото автоматично свойство (property):

```
0 references
internal class Person
{
    0 references
    public int MyProperty { get; set; }
```

Като за нас остава само да редактираме типа и името. В случая, при новия синтаксис { get; set; } записът е много съкратен и включва скрита защита с private в себе си. Той е подходящ за по-прости свойства, които не изискват допълнителни проверки за валидация.

Нека допишем няколко реалистични свойства за клас Person. **Нов синтаксис:**

```
internal class Person
{
    1 reference
    public string FirstName { get; set; }
    1 reference
    public string LastName { get; set; }
    1 reference
    public int Age { get; set; }

    0 references
    public void Identity()
    {
        Console.WriteLine($"First Name: {FirstName, -15} Last Name: {LastName, -15} Age: {Age}");
    }
}
```

Prop + Tab + Tab

1) свойства
- автоматични, публични,
с главна буква

2) действия / методи

Но както казахме, с годините, все повече сложен код се скрива, а видими остават все по-малко функционалности, които са автоматизирани и опростени, но и ограничават редакциите. Нека се върнем стъпки назад да видим представянето в стария синтаксис:

```
internal class Person
{
    private string firstName;
    private string lastName;
    private int age;

    1 reference
    public string FirstName
    {
        get { return firstName; }
        set { firstName = value; }
    }

    1 reference
    public string LastName
    { get { return lastName; } set { lastName = value; } }

    1 reference
    public int Age
    { get => age; set { age = value; } }

    0 references
    public void Identity()
    {
        Console.WriteLine($"First Name: {FirstName, -15} Last Name: {LastName, -15} Age: {Age}");
    }
}
```

1) private field -
частно поле с малка
буква (защита,

2) public property -
публични свойства
главна буква

характеристики /
атрибути

метод / действие / action

Важното е да запомните, че имаме частно поле private field с малка буква, което е защитата, т.к. е невидимо и недостъпно и публично свойство public property с главна буква, което е посредник.

При новия синтаксис имаме на 1 ред **public string FirstName { get; set; }**

При стария синтаксис имаме следния по-дълъг запис:

```
private string firstName;
public string FirstName
{
    get { return firstName; }
    set { firstName = value; }
}
```

Обръщам внимание, че вътре в public с главна буква, get + set важат за капсулирането на private с малка буква.

В този си вид, двата записа са напълно еквивалентни, просто по-съкратеният е с по-нов синтаксис, докато старият е по-подробен. Целта е била да се влезе в тези get и set, за да се направят валидации. Показала съм още 2 синтаксиса – на 1 ред (който не е желателен, но се среща при автодописване) и с get => който е в още по-нова версия на .net 7.0 и след нея, но няма да го използваме.

Характеристика	<code>get { return x; }</code>	<code>get => x;</code>
Синтаксис	<code>{ }</code> блок	<code>=></code> оператор
Брой редове	Може да е многоредов	Винаги един ред
Сложна логика	✓ Подходящ	✗ Неподходящ
Четимост	Добра за сложен код	Отлична за прост код
Версии на C#	Всички версии	C# 7.0+

get – вземам – само за четене (read-only) - id

set – поставям – само за запис (write-only) - passwords

Възможно е да имаме само get или само set.

Нека сега валидираме чрез set какво може да се записва. Реално просто добавяме по един ред ако има грешка, да изпише съобщение. В първия случай проверката е най-проста, а в другите се хвърля съобщение за изключение.

```
public string FirstName
{
    get { return firstName; }
    set { if (firstName.Length == 0) Console.WriteLine("Не сме въвели име!");
          else firstName = value; }
}

1 reference
public string LastName
{
    get { return lastName; }
    set { if (string.IsNullOrEmpty(value)) throw new ArgumentException("Empty field!");
          else lastName = value; }
}

1 reference
public int Age
{
    get { return age; }
    set { if (age < 0 || age > 120) throw new ArgumentException("Invalid age!");
          else age = value; } }

// null или string.Empty свикнали сме
// по-професионален
```

```
public string FirstName
{
    get { return firstName; }
```

```

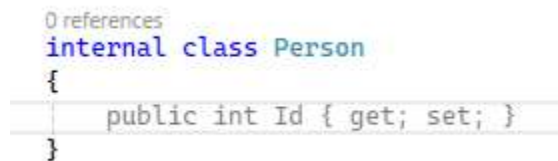
        set { if (firstName.Length == string.Empty) Console.WriteLine("Не сте
въвели име!");
            else firstName = value; }
    }
    public string LastName
    {
        get { return lastName; }
        set { if (string.IsNullOrEmpty(value)) throw new
ArgumentOutOfRangeException("Empty field!");
            else lastName = value; }
    }
    public int Age
    {
        get { return age; }
        set { if (age < 0 || age > 120) throw new
ArgumentOutOfRangeException("Invalid age!");
            else age = value; } }

```

Сега може да създадем и няколко обекта с различни характеристики в Main().

Може да задаваме различни валидации в set, например, че температурата на стаята може да бъде само между -5 и +45 градуса.

Да се върнем на автоматичните свойства за дописване. Казахме, че класът може да се използва за модел (шаблон) за създаване на различни обекти и е желателно всеки от тях да си има ID. Обикновено, ID не е предназначено за редакция – то е само за четене.



```

0 references
internal class Person
{
    public int Id { get; set; }
}

```

Only GET = read-only

```
public string ID { get; } = "007";
```

ИЛИ

```

private string id = "007";

public string ID
{
    get { return id; }
}

```

Only SET = write-only

```

private string password;

public string Password
{
    set { password = value; }
}

```

Може да разгледате упражнителния сайт: https://www.w3schools.com/cs/cs_oop.php

Задача: създайте класове, с техни свойства, методи и обекти по образците, които са предложени:

The screenshot shows the W3Schools website with the C# OOP page. The left sidebar has a menu with 'C# OOP' highlighted. The main content area has the text 'Look at the following illustration to see the difference between class and objects:'. Below this is a diagram with two boxes. The left box is titled 'class' and contains the word 'Fruit'. The right box is titled 'objects' and contains a list of fruit names: 'Apple', 'Banana', and 'Mango'.

The screenshot shows the W3Schools website with the C# OOP page. The left sidebar has a menu with 'C# OOP' highlighted. The main content area has the text 'Another example:'. Below this is a diagram with two boxes. The left box is titled 'class' and contains the word 'Car'. The right box is titled 'objects' and contains a list of car brands: 'Volvo', 'Audi', and 'Toyota'.

So, a class is a template for objects, and an object is an instance of a class.